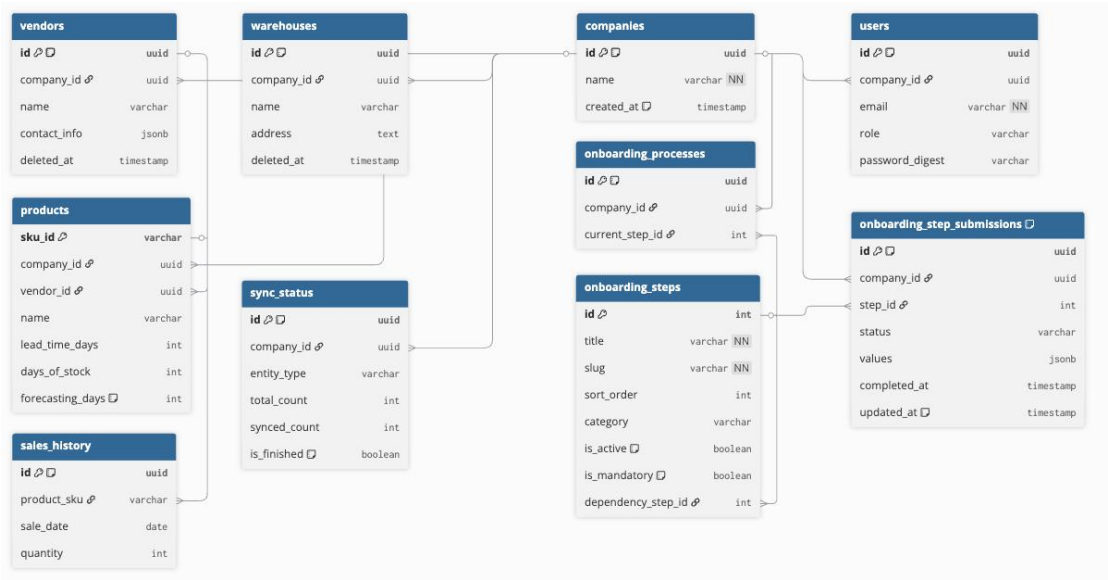
PART 1: Onboarding System Architecture Design (Rails Onboarding Project)

Data Model Design

https://dbdiagram.io/e/6996f7e0bd82f5fce22d8630/6996f8e3bd82f5fce22da298



Justifications for the design choices

- **Multi-Tenant Isolation**

Tables include a company_id. This is a classic Multi-tenant architecture.

It ensures data security and easy filtering. By partitioning data by company_id, you prevent "data leakage" between different clients and simplify the logic for your API queries.

- **State-Machine Onboarding Logic**

separation of onboarding_steps from onboarding_step_submissions (progress) is scalable.

Sequential Control: The dependency_step_id allows you to create a "locking" mechanism, ensuring a company can't set their forecasting_days until they've synced their products.

Flexible Data Capture: Using jsonb for the values column in submissions is a smart move. Different onboarding steps will require different inputs (e.g., a number for forecasting days vs. a list of warehouse IDs); JSONB handles this without requiring a schema migration for every new step.

- **Asynchronous Data Handling**

The sync_status table suggests an Event-Driven or Background Job architecture.

UX Transparency: Since syncing thousands of sales_history rows or products via API can take time, this table allows the frontend to show a progress bar (synced_count / total_count).
Resilience: It helps track where a sync failed, allowing for easier retries.

- **Relational**

UUIDs for PKs: Using uuid for primary keys (like companies.id) is a best practice for distributed systems and security, as it makes IDs unguessable and prevents revealing your total customer count via URL parameters

- **Historical Data Granularity**

By storing sale_date and quantity at the SKU level, you provide the raw data necessary for the RefreshCalculationsWorker to generate accurate demand forecasts.

Core structure you can find on docs file project **[part-1-backend-ruby-api/dbdiagram]**

Tables Justification

Core Identity & Onboarding Tables

Table	Purpose	Justification
companies	The root tenant. All data is scoped to a company UUID.	Ensures strict data isolation for security and compliance.
users	Individual accounts linked to a company.	Supports role-based access control and tracks which user completed which step.
onboarding_steps	Static definitions of the guide	Decoupling definitions from progress allows us to add or reorder steps without data migrations.
onboarding_processes	High-level "state" tracker for a company's journey.	Provides a single source of truth for the frontend to calculate "% complete" and the "Current Step."
onboarding_step_submissions	Stores user inputs (e.g., {"days": 63}) and status history.	**JSONB** usage ensures we can store integers, strings, or file paths without changing the schema for every step.

Inventory & Business Logic Tables

Table	Purpose	Justification
sync_status	Tracks the real-time progress of background data fetching.	Drives the "locking" mechanism: Step 5 is locked if sync_status for Products is not finished.
products	The central inventory unit	Fields like lead_time_days are updated via **side effects** when onboarding steps are saved.
warehouses	Physical or virtual stock locations.	Linked to products to track stock-on-hand per location.
vendors	Suppliers for the products.	Critical for Step 6: "Match suppliers," where we link vendor_id to specific products.
sales_history	Time-series data of past quantities sold.	Linked via product_sku. Used by the RefreshCalculationsWorker to generate forecasts.

API Endpoint Design and documentation.

You can run application and see whole documents for a REST API what we can use.

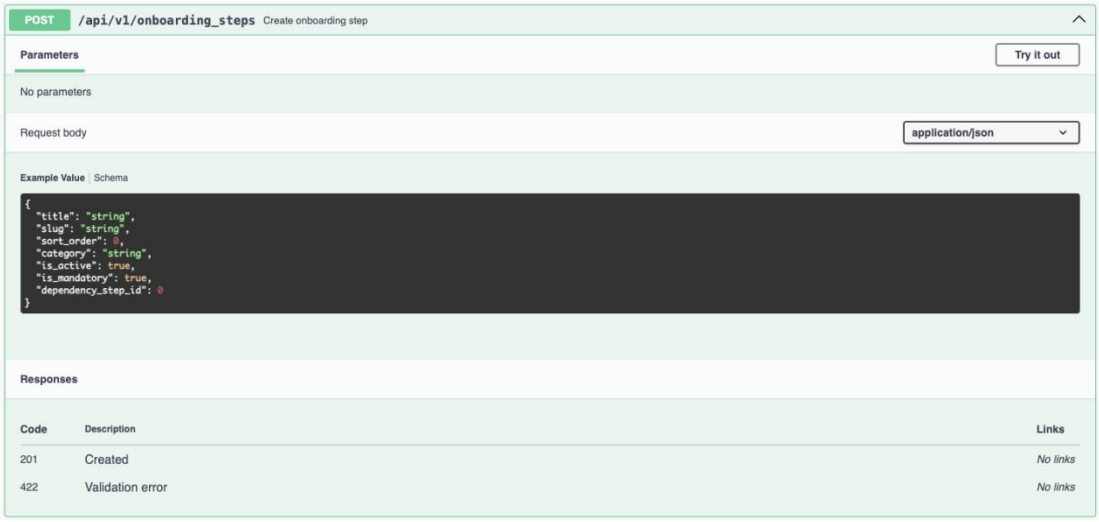
For run and see generated DOCS.
Use a command

---

docker compose up --build -d

after that just visit http://localhost:3000/api-docs

Swagger API DOCS



Background Processes:

My sugestion to use Sidekiq PRO for Production, but for test & development enought to use sidekiq open-source version.

There are a few worker sample of system

1. **OnboardingStepCompletedWorker**
Purpose: Acts as an Event Dispatcher when a user finishes a step.

ustification: Onboarding steps often trigger emails, update state in third-party systems, or recalculate global settings. Moving this out of the Request/Response cycle ensures the user doesn't see a "spinning wheel" after clicking "Submit."

The Handler Pattern: You’re using OnboardingStepCompletionHandler. This is a clean design choice because it encapsulates "Step-Specific" logic (e.g., if step is set_forecasting, trigger calculations) without cluttering the worker.

2. **RefreshCalculationsWorker**
Purpose: The Compute Engine for your inventory intelligence.

Justification: Calculating forecasts for thousands of SKUs involves scanning the sales_history table. This is CPU and I/O intensive and must never happen on the main web thread.

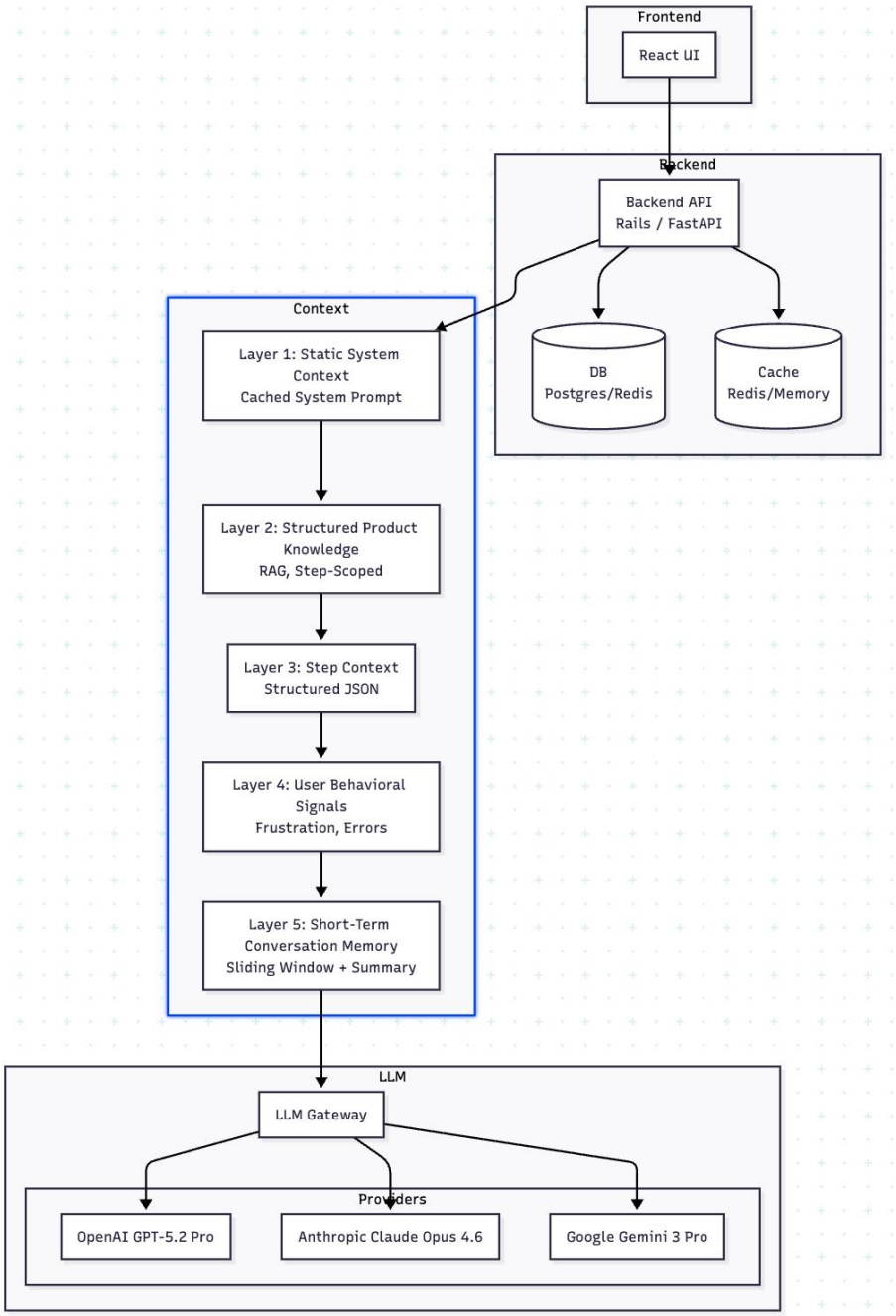
Part 2: AI Integration (Design Exercise)

System Overview
Core Objectives

1. Understand user role
   - Ensures guidance is tailored to permissions (trial user, etc.)
   - Prevents users from seeing instructions they cannot act on
2. Understand current onboarding step
   - Enables step-aware recommendations
   - Reduces irrelevant suggestions and token pollution
3. Provide step-aware contextual assistance
   - Maps exactly to Layered Context Injection strategy
   - Ensures only relevant context reaches the LLM
4. Maintain memory without overloading token context
   - Sliding window + summarization prevents token overflow and LLM degradation
   - Limits cost & preserves performance
5. Provide only accurate, verifiable information and correct product instructions. If unsure, explicitly state uncertainty rather than guessing
   - Directly aligns with hallucination mitigation
   - Encourages LLM to respond: "I don't have enough information" when uncertain
6. Scale cost-efficiently
   - RAG scoping, caching, token limits, multi-layer summarization
   - Ensures predictable AI usage costs across many customers

High-Level Architecture

1. **Frontend → Backend → Context → LLM → Providers**
   - React UI triggers onboarding step events
   - Backend orchestrates data retrieval + context construction
2. **Context Layer**
   - Layered Context Injection ensures only relevant, structured info reaches the LLM
   - Summarization and token budget control are included (Layer 5 + RAG in Layer 2)
3. **LLM Layer**
   - Gateway handles prompt construction, multi-provider failover, and structured JSON enforcement
   - Supports OpenAI GPT-5.2 Pro, Claude Opus 4.6, and Google Gemini 3 Pro
4. **Database & Cache**
   - Stores business data, behavioral signals, and cached context chunks to optimize latency and cost



picture-AI-1

Context Management Strategy

Architectural Principle

LLMs are stateless and probabilistic.

Therefore:

- must never be treated as a source of truth.
- must not receive uncontrolled context.
- must operate on curated, structured, layered input.
- Predictability > Creativity.
- Determinism > Verbosity.

Layer 1 — Static System Context(Immutable & Cached)
Purpose: Defines the assistant's identity, constraints, and behavioral rules.

Never changes per request, Is cached, Is reused across sessions, Enforces safety and predictability

Contains:
Assistant identity,Tone of voice, Guardrails,Output format contract, Hallucination constraints, Product knowledge boundaries

Example:
Smart Onboarding Guide for ProductName.
Rules:
- Only provide guidance relevant to the current onboarding step.
- Never invent product features.
- Do not describe UI elements not explicitly provided.
- If unsure, respond with: "I don't have enough information."
- Keep responses short, clear, and actionable.
- Output must follow the required JSON schema.

- **Layer 2 — Structured Product Knowledge(Scoped RAG)**
  Purpose: Inject only the minimum necessary product knowledge.
  Retrieval-Augmented Generation (RAG), Step-scoped retrieval, Vector database with semantic indexing

  Rules:
  Inject only 1–3 relevant passages, No full documentation dumps, No global product injection, No knowledge from unrelated onboarding steps

  Layer 3 — Step Context (Structured, Deterministic)

  Structured JSON of current step:

```
{
  "step_id": "setup_first_product",
  "step_title": "Set Up Your First Product",
  "user_completed_steps": ["create_account"],
  "role": "admin",
  "account_type": "startup_trial"
}
```

  Passes unambiguous structured input, not natural language

- **Layer 4 — User Behavioral Signals**
  Purpose: Adapt assistance based on real-time friction signals, error count, rage clicks, retries
  Example:
```
{"frustration_level": "high",
  "error_count": 3,
  "retry_attempt": true}
```

- **Layer 5 — Short-Term Conversation Memory**
  Purpose: Maintain continuity without token overload.

  Strategy
  Store last 3 interactions, Intent classification, Resolution status, Sliding window memory, Summary compression for older exchanges

[Examples Onboarding new company]

1) “Set Up Your First Product”
Step: Add your first product to the system
Guidance: “Add SKU, name, category, and supplier info. Don’t worry about completing all details now; you can update later.”

2) “Configure Shipping Preferences”
Step: Enable domestic/international shipping and set default warehouse
Guidance: “For your warehouse locations, consider enabling shipping options to match supplier regions. Start with one preferred carrier.”

3) “Sync Inventory Data”
Step: Import CSV or connect supplier feed
Guidance: “Syncing inventory first helps the system recommend optimal restock levels and lead times. You can always re-sync later.”
4) “Invite Team Members”
Step: Add users and assign roles
Guidance: “Start with admins and primary users. Roles define what each team member can see and do.”

5) “Set Up Notifications”
Step: Choose how you receive alerts
Guidance: “Enable low-stock alerts and shipment notifications to stay informed without checking the dashboard constantly.”

Prompt Design Strategy

as Smart Onboarding Assistant
Rules:
1. Only provide guidance for the CURRENT onboarding step.
2. Do not explain unrelated features.
3. Use short, clear instructions.
4. If user is frustrated, simplify and provide step-by-step bullet points.
5. Never invent UI elements that are not in the provided context.
6. If missing data — ask one clarifying question.
7. Output must be structured in JSON:

```
{
  "message": "...",
  "suggested_actions": [],
  "escalation_needed": false
}
```

Edge Case

1. User Asks About Future Step

-- Asks: “How do I invite my team?”

Solution:

Detect mismatch

Respond:

"Inviting your team happens in Step 4. Let's complete this step first."

2. User Goes Off-Topic

-- Ask: “What's your pricing model?”

Strategy:

Route to FAQ model or different assistant

Do not pollute onboarding context

3. Hallucination Risk

-- ASK: ““Click the blue advanced button”

Prevention:

Allowed actions whitelist

Instruction constraint:

“Only reference UI elements provided in context”

4. Role-Based Context Drift

Remove admin-only features from injected knowledge

5. Repeated Failure

error_count > 3

time_on_step > threshold

System triggers:

Live chat

Email support

Guided walkthrough mode

6. Long-Term Memory Contamination

We never store:

Sensitive tokens

OAuth secrets

Full logs

Only abstracted state.

Optimization:

Use GPT-4 only for complex cases

Default to GPT-4o-mini for simple help

Cache similar error responses

Compress memory aggressively